



POLITECNICO MILANO 1863

Cycle Slip Simulation and Identification

Code Repository: https://github.com/AriannaBoisseau/geoinfo_project

Website: <https://cycle-slip-simulation.streamlit.app/>

Authors:

Arianna Boisseau, 10682839, arianna.boisseau@mail.polimi.it

Edoardo Pessina, 10713416, edoardo3.pesina@mail.polimi.it

Delivery Date: 02/02/2026

Course name: Geoinformatics Project

Course responsible: Mariagrazia Fugini

Project supervisors: Ludovico Biagi, Nikolina Zallemei

Academic Year: 2025–2026

Contents

1	Introduction	2
2	Project Development	3
2.1	Architecture	3
2.2	Key Features	3
2.3	Web Deployment	4
2.4	Tools	4
3	Signal simulation	5
3.1	Satellite parameters	5
3.2	Phase simulation	5
3.2.1	Distance	5
3.2.2	Clock offset	6
3.2.3	Noise	6
3.2.4	Integer ambiguity	6
3.2.5	Ionospheric correction	6
3.2.6	Simplifications	6
3.3	Dual-Frequency observation	7
3.4	Cycle Slip Simulation	7
4	Cycle Slip Detection Method	8
4.1	Geometry-Free Linear Combination	8
4.2	Cycle Slip Identification	8
5	User Workflow	9
5.1	Parameter Configuration	9
5.2	Simulation Execution	11
5.3	Results Visualization	11
5.4	Data Export	12
6	Case study and Results	13
6.1	Monolithic Processing Workflow	13
6.1.1	Satellite Position and Visibility Analysis	13
6.1.2	Elevation and Distance Metrics	14
6.2	Observation Generation Process	14
6.2.1	Phase Observation Computation	15
6.2.2	Observation Plots from Main Processing	15
6.3	Cycle Slip Injection and Detection	16
7	Conclusions	18

List of Figures

1	Preview	9
2	Sidebar access	9
3	Choice of input method	9
4	Example of parameters that can be set	10
5	File uploading button and template	10
6	File uploaded and parsed correctly	11
7	Map showing receiver location and computation button	11
8	Download CSV buttons	12
9	Polynomial clock offset	14
10	Trajectory of the satellite in 24 hours	14
11	Visibility time window	15
12	Elevation and Distance	15
13	L1 and L2 observations	16
14	Differences between consecutive epochs in L1	16
15	Differences between consecutive epochs in L2	16
16	Difference of differences (note: different scales are used)	16

1 Introduction

This project focuses on simulating and analyzing dual-frequency satellite observations from the ground station perspective, with particular emphasis on cycle slip detection and user parametrization. By implementing a complete satellite orbit propagation model, we simulate the position of a satellite over a 24-hour period and analyze its visibility and observability in order to generate realistic GNSS carrier phase observations. The tool allows users to inject artificial cycle slips into the observations and evaluate detection algorithms using a geometry-free linear combination approach.

The foundation of our work lies in the accurate computation of satellite trajectories using Keplerian orbital mechanics. Starting from almanac data containing orbital elements, we propagate satellite positions through successive transformations — from the Orbital Reference System (ORS) to the International Terrestrial Reference Frame (ITRF) — accounting for time-dependent variations in the argument of perigee, inclination, and right ascension of the ascending node. This approach enables us to generate precise three-dimensional coordinates at one-second intervals throughout the day.

A primary objective of this project is the robust detection of cycle slips, sudden integer jumps in carrier phase measurements that result in a temporary loss of signal lock. These discontinuities, often triggered by signal obstructions or ionospheric turbulence, are a critical bottleneck in high-precision GNSS positioning. Because carrier phase observations are ambiguous by an integer number of cycles, any rupture in signal continuity corrupts the satellite-to-receiver range data, necessitating complex re-initialization of the ambiguity resolution process.

To evaluate detection strategies, we simulate dual-frequency GNSS signals on the L1 and L2 carrier frequencies. By generating synthetic observations that capture realistic geometric and atmospheric conditions, we created a laboratory environment in which controlled cycle slips can be introduced and analyzed. In this context, the ground station in Toamasina, Madagascar, serves as a specific geographic testbed to validate the algorithm's performance under simulated real-world visibility constraints. This framework allows for a rigorous assessment of how these discontinuities impact positioning solutions.

2 Project Development

The software we developed is an interactive web-base application for simulating GNSS signals and analyzing cycle slip phenomena in carrier phase observations. The tool is built with Streamlit, providing an intuitive graphical user interface that enabled users to configure simulation parameters, generate synthetic data and visualize the results by means of plots. The application has been deployed and is publicly available at this link allowing users to access the tool directly from their web browser without any local installation requirements.

2.1 Architecture

The application is structured around several key modules:

- **User Interface (gui.py)**: The main Streamlit application that orchestrates the user experience and visualization components
- **Sidebar Configuration (sidebar.py)**: Provides flexible parameter input through manual entry or JSON file upload
- **Core Computations (functions.py)**: Contains the mathematical implementations for satellite orbit propagation, coordinate transformations, and observation modeling
- **Simulation Engine (main_functionalities.py)**: Orchestrates the complete simulation workflow from satellite position computation to observation generation
- **Visualization (plots.py)**: Generates comprehensive plots for satellite trajectories, visibility windows, and observation analysis

2.2 Key Features

Following are the key features of our implementation:

1. **Flexible Parameter Input**: Users can configure simulation parameters through two methods:
 - **Manual Entry**: Interactive input fields organized into logical categories (Ground station coordinates, satellite parameters, frequency settings, ground station configuration, ionospheric model parameters, etc.)
 - **File Upload**: JSON file upload for batch processing or predefined scenarios, following the structure defined in `default_parameters.json`
2. **Satellite Orbit Propagation**: The tool implements Keplerian orbit mechanics to compute satellite positions over time. Using orbital elements (eccentricity, semi-major axis, inclination, etc.), the system calculates satellite positions in the International Terrestrial Reference Frame (ITRF).
3. **Visibility Analysis**: The application determines satellite visibility from a user-defined ground station location by:
 - Computing elevation and azimuth angles for each epoch
 - Applying configurable minimum elevation angle thresholds
 - Visualizing the ground station location on an interactive map
 - Filtering observations to include only visible satellite passes
4. **GNSS Observation Simulation**: The tool generates realistic carrier phase observations on two frequencies by modeling the following components:
 - **Geometric Range**: Direct line-of-sight distance from satellite to receiver based on computed orbital positions
 - **Clock Offsets**: Receiver and satellite clock biases with polynomial drift models to simulate realistic timing errors
 - **Integer Ambiguities**: Fixed to a constant value for each satellite-receiver pair to simulate carrier phase observations

- **Measurement Noise:** Configurable Gaussian noise with user-defined standard deviation to simulate realistic observation uncertainty
 - **Ionospheric Delays:** Frequency-dependent signal propagation delays modeled using the Klobuchar algorithm
 - **Frequency-Dependent Effects:** Differential effects across different frequencies to enable dual-frequency analysis techniques
5. **Cycle Slip Simulation and Detection:** The application follows a two steps process:
- **Injection:** Introduces artificial cycle slips at specified epochs by adding integer multiples of the carrier wavelength to simulate realistic measurement discontinuities
 - **Detection:** Using the difference of difference between the two frequency observations, cycle slips are automatically identified
6. **Visualization:** Comprehensive plotting capabilities include:
- Carrier phase observations over time
 - First-order differences for both frequencies
 - Difference of differences
 - Clear identification of cycle slip events in the data
7. **Data Export:** Generated observations can be exported as CSV files for further analysis in external software or integration into processing workflows.

2.3 Web Deployment

The application uses Streamlit's cloud hosting infrastructure, making it accessible worldwide without installation barriers. This web-based deployment offers several advantages:

- **Accessibility:** Users can access the tool from any device with a web browser
- **No Setup Required:** Eliminates the need for local Python environment configuration
- **Immediate Updates:** Changes and improvements are instantly available to all users

2.4 Tools

The project was entirely developed in python, this are the libraries that were used:

- **numpy:** Used for numerical computations, including array manipulations and mathematical functions essential for orbit propagation and observation modeling
- **pandas:** Utilized for data handling and manipulation, particularly in structuring GNSS observations or importing from CSV format.
- **streamlit:** The primary framework for building the web application, providing tools for user interface components, interactivity and deployment.
- **scipy:** Employed for values of numerical constants like e
- **matplotlib:** used for generating static plots and visualizations of GNSS observations

3 Signal simulation

3.1 Satellite parameters

To simulate the phase observation we used the almanac of the satellite SVN 63 for the 28th November 2016. This are the values used to compute the satellite orbit:

- $\sqrt{a} = 5.153650835037 \times 10^3$ [m], square root of semi-major axis
- $e = 3.841053112410 \times 10^{-3}$ [m], eccentricity
- $M_0 = 295004883409$ [rad], mean anomaly
- $\Omega_0 = -2.241692424630 \times 10^{-1}$ [rad], longitude of the ascending node
- $\dot{\Omega} = -8.386063598924 \times 10^{-9}$ [rad/s], rate of node's right ascension
- $i_0 = 9.634782624741 \times 10^{-1}$ [rad], inclination
- $\dot{i} = -7.286017777600 \times 10^{-11}$ [rad/s], rate of inclination angle
- $\omega_0 = 9.419793734505 \times 10^{-1}$ [rad], argument of perigee
- $\dot{\omega} = 0$ [rad/s], rate of argument of perigee

3.2 Phase simulation

This is the formula that we used for the simulation of the phase observation. The output is given in meters:

$$\Phi = \rho + c * t_r - I_f + \epsilon + N * \lambda_{wl} \quad (1)$$

where:

- Φ is the phase observation
- ρ is the distance from the satellite to the user
- t_r is the receiver clock offset
- c is the speed of light in vacuum
- I_f is the ionospheric delay at frequency f
- ϵ is a random gaussian noise
- N is the integer ambiguity
- λ_{wl} is the wavelength of the carrier

3.2.1 Distance

The geometric distance from the satellite to the ground station is computed by first determining the satellite position in ITRF coordinates system through the Keplerian orbit propagation. Starting from the almanac parameters, we computed the satellite's position at each epoch, then calculated the three-dimensional Euclidian distance:

$$\rho = \sqrt{(X_s - X_r)^2 + (Y_s - Y_r)^2 + (Z_s - Z_r)^2} \quad (2)$$

where (X_s, Y_s, Z_s) are the coordinates of the satellites and (X_r, Y_r, Z_r) are the coordinates for the ground station in ITRF.

3.2.2 Clock offset

The clock offset is modeled using a polynomial function with these three parameters:

- $dt_0 = -7.661711424589 \times 10^{-5}$, satellite clock offset
- $dt_1 = -3.183231456205 \times 10^{-12}$, satellite clock drift
- $dt_2 = 0$, satellite clock drift rate

The clock offset at epoch t is computed as:

$$t_s(t) = dt_0 + dt_1(t - t_r) + dt_2(t - t_r)^2 \quad (3)$$

3.2.3 Noise

To simulate realistic measurement condition, we added a gaussian noise to phase observations following a normal distribution: $\mathcal{N}(0, 1)$ in meters.

3.2.4 Integer ambiguity

Integer ambiguity was set to 100 for all epochs in the simulation. This parameters remains constant through each observation and represents the unknown number of complete carrier wavelengths in the initial satellite-receiver distance.

3.2.5 Ionospheric correction

The Ionosphere is a dispersive medium that affects GNSS signals differently depending on their frequency. To simulate realistic observation conditions, we implemented the Klobuchar ionospheric model, which provides a computationally efficient method for the users to mitigate ionospheric delay. The Klobuchar model uses eight parameters to describe the amplitude and period of a cosine function that approximates the vertical ionospheric delay. The model computes the slant ionospheric delay as a function of:

- User position (latitude and longitude)
- Satellite elevation and azimuth angles
- Local time at the ionospheric pierce point
- Geomagnetic latitude

The ionospheric delay is frequency-dependent and scales with the inverse square of the carrier frequency:

$$I_f = I_{L1} * (\frac{f_{L1}}{f})^2$$

where I_{L1} is the ionospheric delay on the L1 frequency, and f is the frequency of interest.

The Klobuchar standard model is used to simulate the ionospheric disturbance in L1. Klobuchar parameters are obtained by official navigation data:

$$I_{L2} = I_{L1}(\frac{f_{L1}}{f_{L2}})^2$$

This frequency-dependent ionospheric delay is incorporated into the phase observation equation, adding realism to the simulated measurements.

3.2.6 Simplifications

For this simulation, we have neglected the following effects:

- Satellite's clock offset
- Tropospheric corrections

This simplification are done since these are parameter that will be modeled added and removed to the final observation value.

3.3 Dual-Frequency observation

GPS satellites transmit signals on two primary carrier frequencies in the L-band:

- L1 frequency: $f_1 = 1575.42 \text{ MHz}$
- L2 frequency: $f_2 = 1227.60 \text{ MHz}$

We generate independent phase observations on both frequencies using the formula in Equation 1, with the respective wavelength for each carrier.

3.4 Cycle Slip Simulation

To evaluate cycle slip detection algorithms and to simulate cycle slips in the carrier phase observations, we introduce artificial discontinuities at a user selected epoch. A cycle slip is modeled by adding an integer multiple of the carrier wavelength to the phase observation at the specified epoch. The cycle slip is introduced between two consecutive epochs, affecting all subsequent observations. The magnitude of the cycle slip is randomly chosen but a seed has been set for reproducibility. The cycle slip is applied only to L1 observations using this formula:

$$\Phi_{f,new}(t_{slip}) = \Phi_{f,old}(t_{slip}) + k * \lambda_f \quad (4)$$

where:

- $\Phi_{f,new}(t_{slip})$ is the phase observation after the cycle slip at frequency f
- $\Phi_{f,old}(t_{slip})$ is the phase observation before the cycle slip at frequency f
- k is a randomly chosen integer representing the number of cycles slipped
- λ_f is the wavelength of the carrier at frequency f

4 Cycle Slip Detection Method

To detect cycle slips in the simulated dual-frequency carrier phase observations, we implement a geometry-free linear combination approach. This method leverages the difference of differences between the L1 and L2 observations to identify discontinuities indicative of cycle slips.

4.1 Geometry-Free Linear Combination

We first compute the time differences of the phase observations for both frequencies:

$$\Delta\Phi_f(t) = \Phi_f(t) - \Phi_f(t-1) \quad (5)$$

where f represents either the L1 or L2 frequency.

Next, we calculate the geometry-free linear combination by taking the difference between the first-order differences of the two frequencies.

The difference in time of the geometry-free linear combination is computed as follows:

$$D(t) = (\Phi_{L1}(t) - \Phi_{L1}(t-1)) - (\Phi_{L2}(t) - \Phi_{L2}(t-1)) \quad (6)$$

where $D(t)$ is the geometry-free linear combination at epoch t , $\Phi_{L1}(t)$ and $\Phi_{L2}(t)$ are the phase observations at frequencies L1 and L2, respectively.

$D(t)$ can also be represented as the sum of two components:

- a ionospheric term, should be small with or without cycle slip
- a term that depends on the integer ambiguities, close to zero without a cycle slip, different from zero when a cycle slip occurred

$$D(t) = \left[\frac{1}{f_{L1}^2} - \frac{1}{f_{L2}^2} \right] * [\delta I(t)] + \lambda_{L1} \delta N_{L1} - \lambda_{L2} \delta N_{L2} \quad (7)$$

An high value of $D(t)$ indicates a potential cycle slip event, as it reflects a sudden change in the integer ambiguities between the two frequencies, this is why all values of $D(t)$ grater than a certain threshold are marked as cycle slips.

4.2 Cycle Slip Identification

Cycle slips manifest as abrupt changes in the geometry-free linear combination $D(t)$. To identify potential cycle slip epochs, we analyze the time series of $D(t)$ for significant jumps exceeding a predetermined threshold of 5 cm.

The detection algorithm involves the following steps:

- Compute the geometry-free linear combination $D(t)$ for all epochs
- Identify the epochs with absolute value grater than our threshold

The identified epoch is then reported as detected cycle slip, allowing for further analysis and validation against the known injected slip.

5 User Workflow

In this section, we outline the typical workflow a user would follow to utilize our tool successfully.

Upon accessing the website, a user may encounter a warning message saying "this app has gone to sleep". This happens because Streamlit Cloud puts apps to sleep after a period of inactivity to conserve resources. To wake up the app, the user simply needs to click on the button "Yes, get this app back up!". This action will reactivate the application, allowing the user to proceed with their tasks, a preview of the website will be shown as in Figure 1.

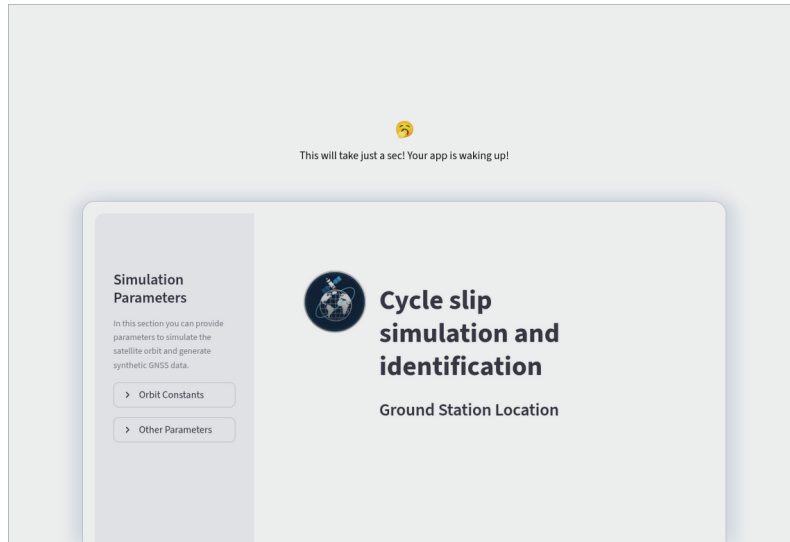


Figure 1: Preview

5.1 Parameter Configuration

The first time a user accesses the application, they are presented with a message inviting them to configure the simulation parameters using the sidebar as shown in Figure 2. The sidebar can be accessed by clicking on the arrow located on the top left of the page, circled in red in Figure 2.

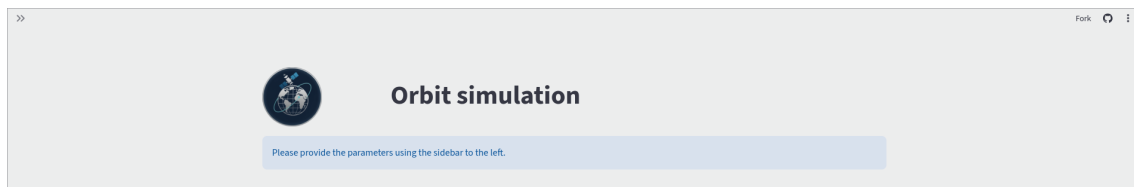


Figure 2: Sidebar access

Once the sidebar is opened, the user is prompted to choose between two methods for setting the simulation parameters: manual entry or file upload. The two buttons can be seen in Figure 3.

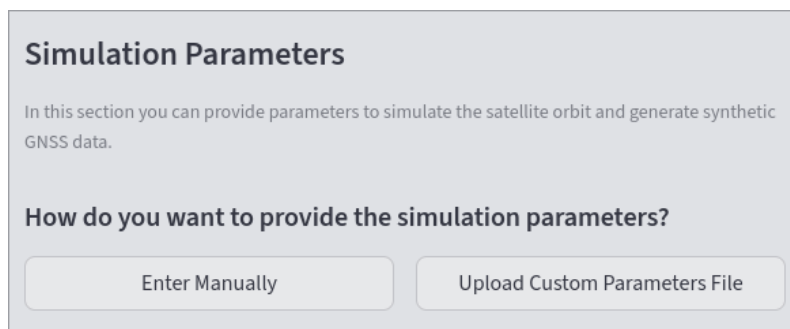


Figure 3: Choice of input method

The user can select either option based on their preference:

- **Manual Entry:** Users can input parameters directly into the provided fields, organized into logical categories such as satellite parameters, frequency settings, ground station coordinates, ionospheric model parameters and others.
- **File Upload:** Users can upload a JSON file containing predefined parameters. The structure of the JSON file should match the format defined in the `default_parameters.json` file provided below the file upload section as a reference.

If the user chooses to set parameters manually, they can fill in the fields displayed in the sidebar, two of this sections are shown in Figure 4 as an example. Each section can be expanded or collapsed for better organization. Each field is already populated with default values, which the user can modify as needed.

(a) Satellite Parameters

Keplerian Orbit Parameters

Eccentricity	3.84105311241e-3	-	+	Sqrt of semi-major axis	5.153650835037e+3	-	+
Mean Anomaly	1.295004883409e+0	-	+	Longitude of the ascending node	-2.24169242463e-1	-	+
Rate of node's right ascension	-8.386063598924e-9	-	+	Inclination	9.634782624741e-1	-	+
Rate of inclination angle	-7.2860177776e-11	-	+	Argument of perigee	9.419793734505e-1	-	+
Rate of argument of perigee	0e+0	-	+				

Other Satellite Parameters

Integer Ambiguity Value	100	-	+
-------------------------	-----	---	---

(b) Ground Station Parameters

Clock Offset

dt0	-7.661711424589e-5	-	+
dt1	-3.183231456205e-12	-	+
dt2	0e+0	-	+
Clock Offset Noise Standard Deviation	1e-3	-	+

Position

Latitude of ground station (degrees)	-18.16	-	+	Longitude of ground station (degrees)	49.41	-	+
Altitude of ground station (meters)	0.00	-	+	Minimum Elevation Angle (degrees)	0.00	-	+

(a) Satellite parameters section

(b) Ground Station Parameters

Figure 4: Example of parameters that can be set

If the user opts for file upload, they can click on the "Browse files" button to select a JSON file from their local machine. An example of the file upload interface is shown in Figure 5. Once a file is selected, the application will parse the JSON content and populate the simulation parameters accordingly, a green message will appear confirming the successful upload as shown in Figure 6.

Upload your own parameters file

Drag and drop file here
Limit 200MB per file • JSON

Browse files

Template for Parameters File

If you choose to upload a custom parameters file, please ensure it follows the structure below:

```
{
  "dt0": -7.661711424589e-05,
  "dt1": -3.183231456205e-12,
  "dt2": 0.000000000000e+00,
```

Figure 5: File uploading button and template

When the parameters are set correctly, a map will be displayed showing the location of the ground station as in Figure 7.

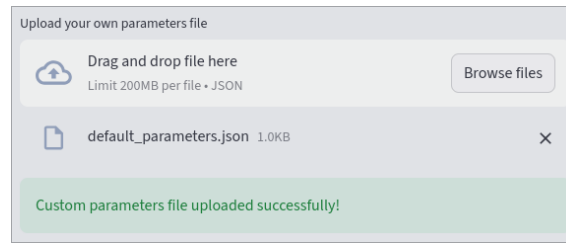


Figure 6: File uploaded and parsed correctly

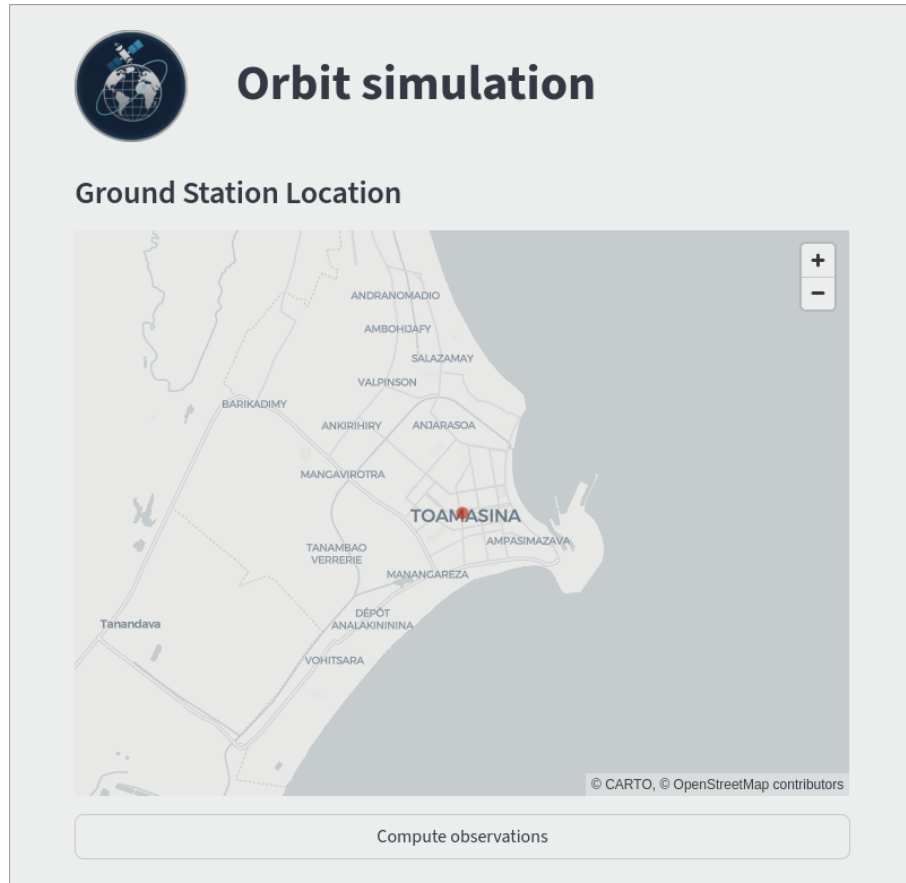


Figure 7: Map showing receiver location and computation button

5.2 Simulation Execution

After having verified the location of the ground station on the map, the user can proceed to run the simulation by clicking on the "Run Simulation" button located below the map as shown in Figure 7. Once clicked, the application will execute the simulation based on the configured parameters, generating synthetic GNSS data and visualizations.

5.3 Results Visualization

Upon completion of the simulation, the application will display a series of plots illustrating the results. These plots include:

- L1 and L2 carrier phase observations over time
- First-order differences for both frequencies
- Geometry-free linear combination (difference of differences)
- Identification of cycle slip events in the data

Examples of these plots can be seen in Section 6.

5.4 Data Export

Finally, the user has the option to export the generated observations as a CSV file for further analysis. A "Download CSV" button is provided below the plots, as shown in Figure 8. By clicking this button, the user can download the synthetic GNSS data in a structured format suitable for external processing or integration into other workflows.



Figure 8: Download CSV buttons

6 Case study and Results

Multiple simulations were conducted to validate the functionality and accuracy of the GNSS cycle slip simulation and detection tool. We present here a detailed case study demonstrating the tool's capabilities and the results obtained.

We simulated the carrier phase observations for GPS satellite SVN 63 on November 28, 2016. The simulation parameters were set to reflect realistic conditions, including satellite orbital elements, ground station location, and ionospheric model settings. The ground station was positioned in the city of Toamasina, Madagascar, with coordinates latitude -18.1553985 and longitude 49.4098352. This location was chosen because the satellite passed directly overhead during the simulation period, providing optimal visibility for observation generation.

6.1 Monolithic Processing Workflow

Initially, the project utilized a single monolithic 'main.py' script that performed all simulation tasks sequentially. The output log from this execution (as documented in 'output/main_output.txt') shows the complete processing pipeline:

```

Importing default parameters and initializing variables ... done.
Computing clock offsets... done.
Computing satellite positions... done.
Converting Cartesian coordinates to Geodetic coordinates... done.
Plotting satellite trajectory with basemap... done.
Cheching observability period in Toamasina... done.
Plotting elevation for visible timewindow...
=====
SATELLITE VISIBILITY ANALYSIS
=====
Receiver Coordinates: -18.1554°, 49.4098°

Number of visibility periods: 1
Total visible time: 26477 seconds (7.35 hours)

Visibility periods:
Start (HH:MM:SS)      End (HH:MM:SS)      Duration (min)
-----
14:51:33              22:12:49            441.3
Min distance: 20158385.7 meters at time 68349 seconds
Max distance: 32840861.5 meters time 30235 seconds
=====
Computing distance between satellite and ground station over time... done.
Exporting dataframes and observations to csv... done.
```

This monolithic approach generated comprehensive plots at each processing stage:

6.1.1 Satellite Position and Visibility Analysis

The first series of plots generated by main.py illustrate the fundamental satellite positioning and visibility analysis:

- **Clock Offset Plot:** Visualizes the polynomial clock offset function over the 24-hour simulation period, showing the drift characteristics modeled by the parameters dt_0 , dt_1 , and dt_2 , Figure 9
- **Satellite Trajectory:** Displays the ground track of the satellite in geographic coordinates, showing the path across the Earth's surface, Figure 10
- **Visibility Window:** Illustrates which epochs the satellite is visible from the Toamasina ground station, determined by the elevation angle threshold, Figure 11

The visibility analysis revealed the following key results from the monolithic execution:

- Number of visibility periods: 1
- Total visible time: 26477 seconds (7.35 hours)

- Visibility period: 14:51:33 to 22:12:49 (Duration: 441.3 minutes)
- Minimum distance: 20158.39 km at epoch 68349 seconds
- Maximum distance: 32840.86 km at epoch 30235 seconds

Here are the plots just mentioned:

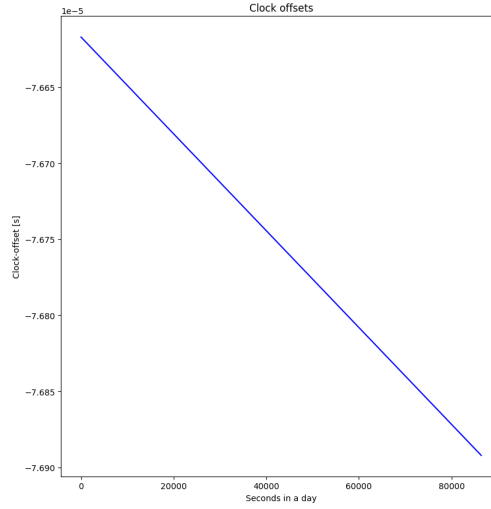


Figure 9: Polynomial clock offset

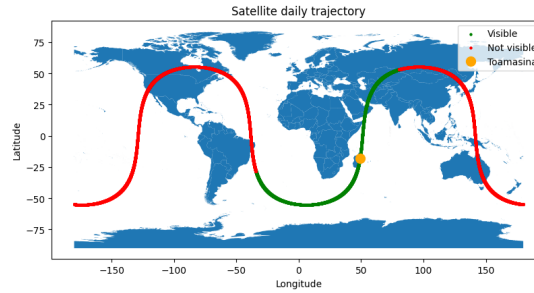


Figure 10: Trajectory of the satellite in 24 hours

6.1.2 Elevation and Distance Metrics

The main.py script generated additional plots showing:

- **Elevation Angle Plot:** Shows the elevation angle variation over time for the visible portion of the satellite pass. The minimum elevation angle threshold of 0 degrees filters out periods when the satellite is below the horizon, Figure 12a
- **Distance Profile:** Illustrates the three-dimensional distance between satellite and ground station, with clear identification of closest approach and maximum distance epochs, Figure 12b

6.2 Observation Generation Process

The simulation was executed for a duration of twenty-four hours, generating dual-frequency carrier phase observations (L1 and L2) at one-second intervals. Only observations during periods when the satellite was visible from the ground station (elevation angle above 0 degrees) were retained for analysis.

The observation generation process followed this sequence:

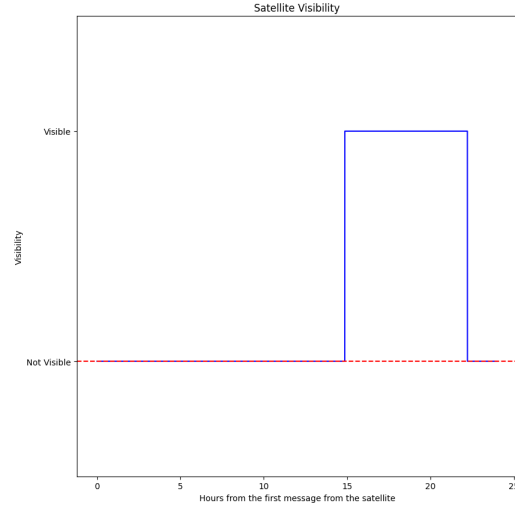


Figure 11: Visibility time window

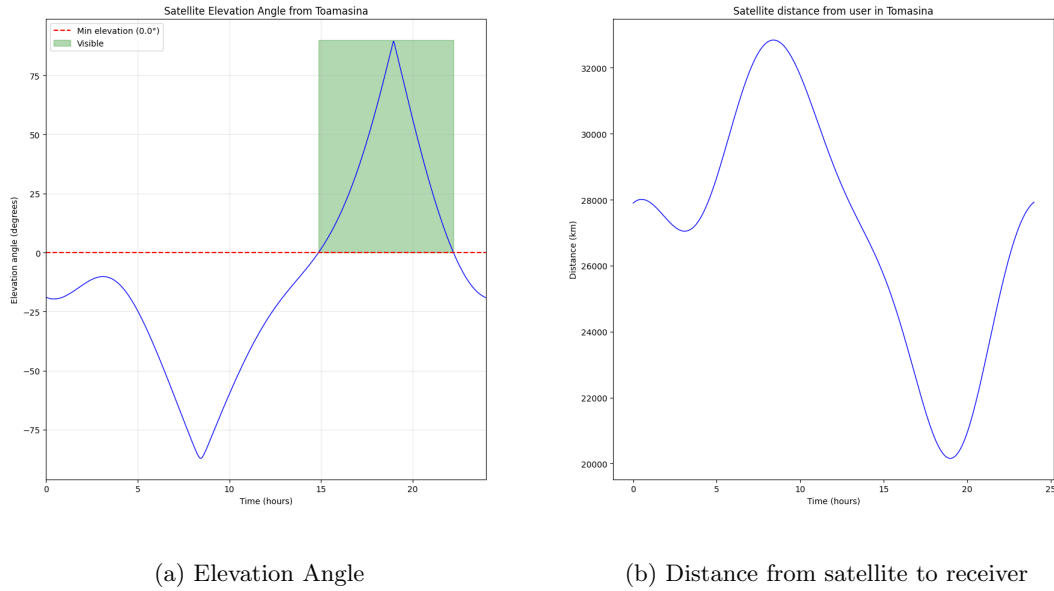


Figure 12: Elevation and Distance

6.2.1 Phase Observation Computation

The main.py script computed observations using Formula 1 where all components (geometric distance, clock offset, ionospheric correction, noise, and integer ambiguity) were computed at each visible epoch.

6.2.2 Observation Plots from Main Processing

The main.py execution generated plots showing:

- **L1 and L2 Observations:** Raw carrier phase observations on both frequencies over the entire visible period, showing the combined effects of range, clock offset, ionospheric delay, noise, and integer ambiguity.
- **Observation Differences:** First-order differences of the phase observations, revealing the temporal variability and noise characteristics, and difference of differences.

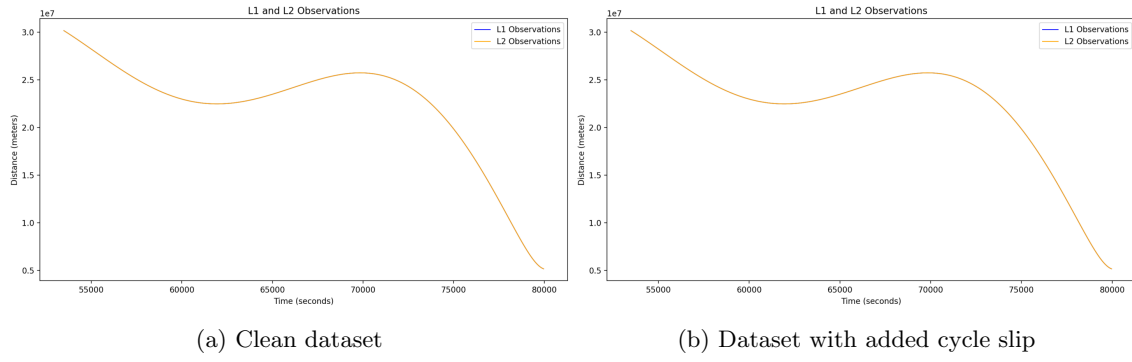


Figure 13: L1 and L2 observations

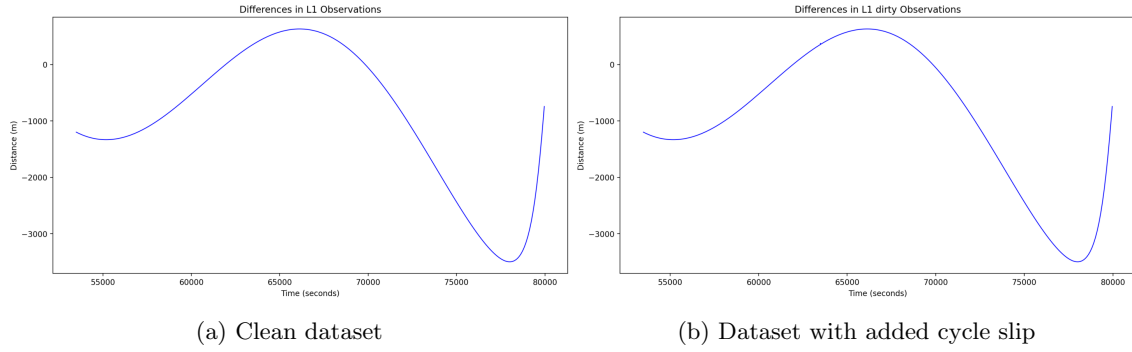


Figure 14: Differences between consecutive epochs in L1

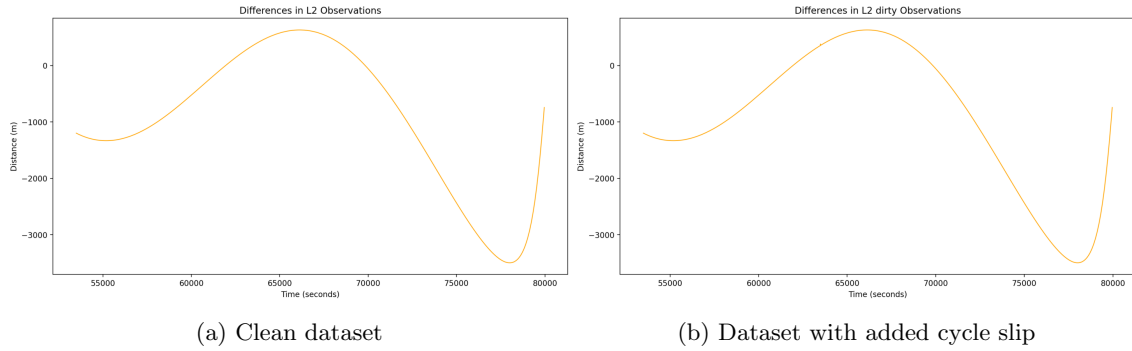


Figure 15: Differences between consecutive epochs in L2

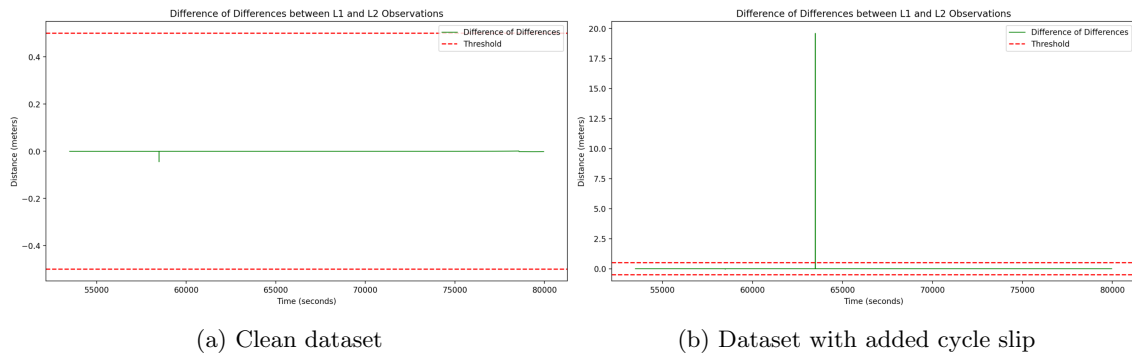


Figure 16: Difference of differences (note: different scales are used)

6.3 Cycle Slip Injection and Detection

A cycle slip was artificially injected at epoch 10000 seconds from the start of the visibility period, corresponding to epoch 63491 seconds in the full simulation timeline. The slip magnitude was randomly selected to be 103 cycles, thus adding a discontinuity of $103 * \text{wavelength}_{L1} = 19.6$

meters and applied to only L1 observations using Formula 4.

The cycle slip detection algorithm successfully identified the injected slip at epoch 63491 by analyzing the epochs where the geometry-free linear combination exceeded the predefined threshold of 5 cm represented in red dashed lines in Figure 16. This demonstrates the effectiveness of the geometry-free linear combination method for cycle slip detection in dual-frequency GNSS observations.

7 Conclusions

This project successfully developed an interactive, web-based tool for simulating dual-frequency GNSS signals and identifying cycle slips in carrier phase observations. By integrating Keplerian orbital mechanics and realistic signal modeling, the application provides a robust environment for GNSS data analysis.

Key Outcomes and Findings:

- **Effective Orbit Propagation:** The system accurately modeled satellite trajectories and identified visibility windows for a user selected ground station.
- **Robust Detection Methodology:** The implementation of the geometry-free linear combination method proved highly effective. By analyzing the "difference of differences" between L1 and L2 frequencies, the tool successfully identified artificial cycle slips injected into the dataset.
- **User-Centric Design:** The deployment of the tool via Streamlit ensures global accessibility and ease of use, allowing users to configure complex parameters manually or via JSON uploads without requiring a local Python environment.

In summary, the project provides a comprehensive framework for understanding GNSS signal discontinuities. The tool's ability to simulate "clean" and "dirty" datasets (those with injected slips) serves as a valuable resource for testing and validating cycle slip detection algorithms in a controlled, reproducible environment.